

zkVM Standards & Language Support

Marcin Bugaj • Ethereum Foundation

Two Work Streams

1. zkVM Interface Standards (v0)

Standardizing how zkVMs integrate with execution clients

2. Language Support via WASM

Enabling Go, C#, ... for bare metal zkVM targets

The Integration Challenge

Execution client teams faced a fragmented landscape:

- Each zkVM requires custom compilation targets
- Different precompile interfaces per implementation
- Unique IO handling for each zkVM

Supporting 3 zkVMs = potentially 3 separate integrations

zkVM Standards v0: Three Standards

Created in collaboration with multiple zkVM and client teams

1. RV64IM Target Architecture + Zicclsm Extension

Common instruction set for RISC-V compatible zkVMs

2. C Interface for Precompiles

Standardized API for calling zkVM precompiles

3. C Interface for IO

Standard way to access private inputs and commit to public outputs

Standard 1: RV64IM + Zicclsm

RV64IM: Common ISA baseline

- 64-bit RISC-V with integer multiplication/division
- All RISC-V compatible zkVMs can target this

Zicclsm Extension: Misaligned access support

- Pragmatic safety net for compiler mistakes
- Prevents subtle toolchain differences from becoming consensus issues

Benefit: Optimizations and tooling improvements benefit the entire ecosystem

Standard 2: C Interface for Precompiles

The Problem:

- EVM precompiles are expensive to prove
- Each zkVM implements them as optimized zkVM-specific precompiles

The Solution:

- C-based API for calling zkVM precompiles
- Write precompile-calling code once, switch zkVM backends without refactoring

Example precompiles: keccak256, U256 arithmetic, elliptic curve operations

Standard 3: C Interface for IO

Guest programs need to communicate with the host:

- Receive private input data from the host
- Commit to public outputs that verifiers can check

The standard defines:

- C-based API with two key functions
- Accessing private input data
- Committing to public outputs

Work #2: Language Support via WASM

The Opportunity:

Languages like Go, C#, ... don't target bare metal systems

The Experiment:

Compile: High-level language → WASM → RISC-V64IM bare metal

Question: Is the performance penalty acceptable?

Initial findings: ~3x instruction count overhead vs direct compilation

WASM Benchmarking: Next Steps

Instruction count doesn't tell the full story for zkVMs

Next steps:

- Integrate precompile accelerators (from our standards!)
- Measure actual proving time, not just instruction count

Hypothesis: Different instructions have vastly different proving costs. A precompile invocation may be much more expensive to prove than a simple ADD instruction.

Real overhead might be 1.5x or 1.1x, not 3x.

Impact & Future

Standards work:

- Reduces integration complexity for execution clients
- Enables fair zkVM comparisons and competition
- Foundation for formal verification efforts

Language expansion:

- Could attract developers from Go, C#, TypeScript ecosystems
- Unlock new tooling and optimizations

These are v0 standards - expect evolution as we move toward zkEVM on L1

Thank You

Repositories:

- github.com/eth-act/zkvm-standards
- github.com/eth-act/wasrisc

Questions?